Research Problem: Single-GPU MoE Deployment

- **MoE 的专家是按需调用的参数模块。每次只计算少数专家，但路由器可能选中任意一个。**
  单任务显存和 CPU 内存放不下全部专家时，部分权重留在 SSD；被选中后要先搬运才能计算。

一个请求如何触发冷读取

路由选择

E1	E2	E3	E4	E5	E6	E7	E8

本次选中 E8；蓝色表示被选中的专家

GPU 显存	E1 E2
CPU 内存	E3 E4 E5
SSD 冷驻留	E6 E7 E8

SSD → CPU → GPU

结果：第一个输出词元要等待权重；后续请求也可能重复等候。

典型部署场景

共享 GPU 服务器
每个任务只分到一张卡和部分内存

离线生成任务
问答、代码生成可容忍较长延迟

单卡工作站
模型规模大于可用内存额度

MECC 处理三个决定
先放哪里 · 淘汰后留哪里 · 热点变化后如何更新

目标：在单卡容量约束下部署完整 MoE，并减少专家权重进入计算路径时的等待。

Related Work: Deployment Gaps

- 现有系统已能跨显存、内存和 SSD 按需执行专家，并在推理时保留常用专家。
  共享服务器还需要回答：给定单任务预算如何安排驻留、如何减少初始冷读、
  以及部署前如何估计 SSD 压力。MECC 将这三项能力纳入同一方案。

方法	三级存储支撑	完整专家计算	按预算生成驻留方案	历史热点预热	冷读压力预估	在线适应热点变化
SwapMoE[1]	✗	✗	✗	✓	✗	✓
Fiddler[2]	✗	✓	✗	✗	✗	✗
MoE-Infinity[3]	✗	✓	✗	✗	✗	✓
MoE-Lightning[4]	✗	✓	✗	✗	✗	✗
SSD-LLaMA[5]	✓	✓	✗	✗	✗	✓
MECC	✓	✓	✓	✓	✓	✓

MECC: Three Challenges, Three Decisions

- **单卡部署的难点不是“把权重放在 SSD”这一步，而是让常用专家尽可能少走 SSD 路径。MECC 分别决定初始放置、运行中的保留位置，以及权重何时可以参与计算。**

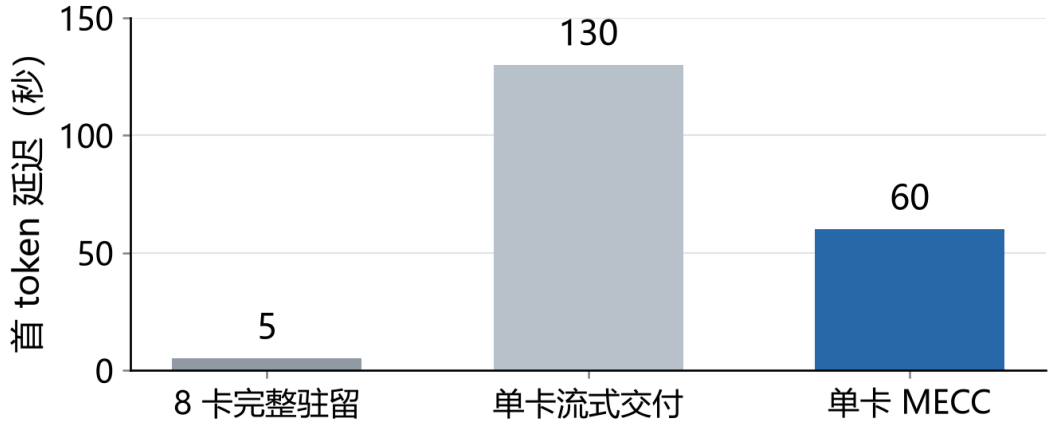
① 冷启动	首个请求可能选中尚未驻留的专家	→	**画像引导初始放置**
② 重复冷读	有限槽位被重复占用或过早淘汰	→	**显存与 CPU 联合保留**
③ 热点漂移	真实任务与部署前画像逐渐不同	→	**按真实路由在线修正**

后续三页先给结果，再解释每个问题为何出现；第 7–9 页给对应洞察。

Motivation I: The First Token Waits for Cold Experts

- 多卡完整驻留时，所选专家已在 GPU；单卡的快层容量有限，首个请求可能选中 SSD 上的专家。图中比较同一 8×7B 问答任务的首 token 延迟，并画出产生等待的权重交付路径。

(a) 首 token 延迟



单卡流式交付的首 token 等待显著更长。

(b) 同一次路由选择的两种部署

多卡：全部专家已分布在 GPU

E1 E2 E3 E4 E5 E6 E7 **E8**

选中 E8 → GPU 显存命中 → 执行

单卡：E8 仍在 SSD

SSD E8 → CPU 缓冲 → GPU 执行

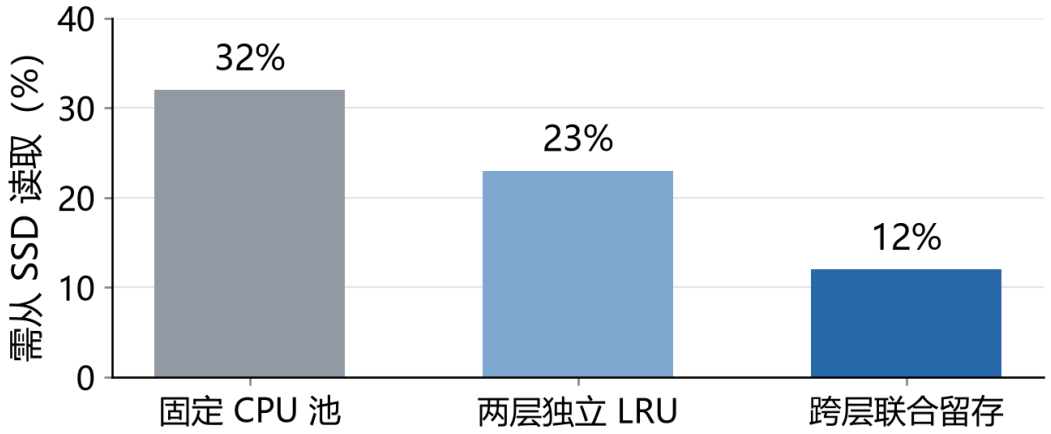
SSD 冷读和 CPU→GPU 搬运位于首 token 的关键路径。

设计问题①：启动前，怎样优先驻留最可能被选中的专家?

Motivation II: Limited Slots Cause Repeated Cold Reads

- **单卡只有有限的显存与 CPU 缓存槽位。若两层分别管理，某些专家会重复占位；另一些被挤到 SSD，之后再次被选中就要冷读。左图比较相同槽位预算下的 SSD 请求比例。**

(a) 需从 SSD 读取的请求占比



联合保留减少了落到 SSD 的请求。

(b) 五个快层槽位能覆盖几个专家

两层独立管理

显存 E1 E2　CPU E1 E3 E4

E1 重复占位，E5 只能留在 SSD

跨层联合保留

显存 E1 E2　CPU E3 E4 E5

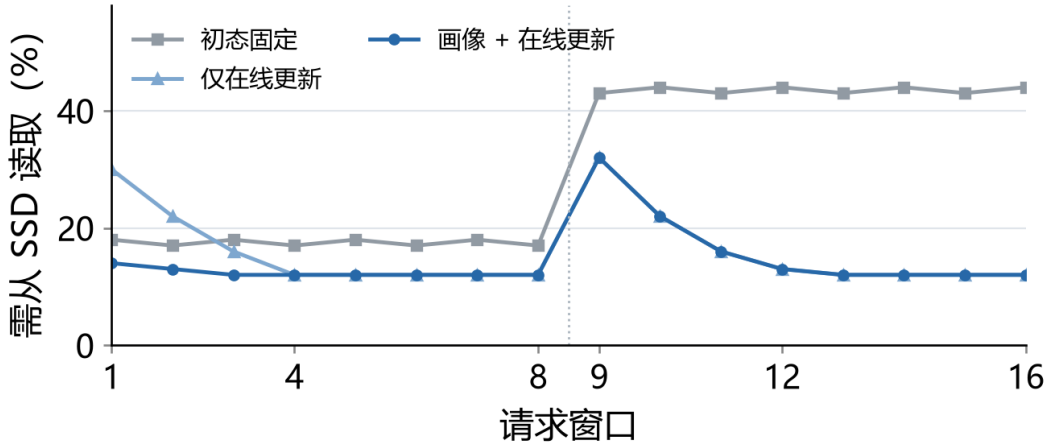
五个槽位覆盖五个不同专家

设计问题②：怎样用相同槽位保留更多不同的可复用专家?

Motivation III: Hot Experts Change with the Task

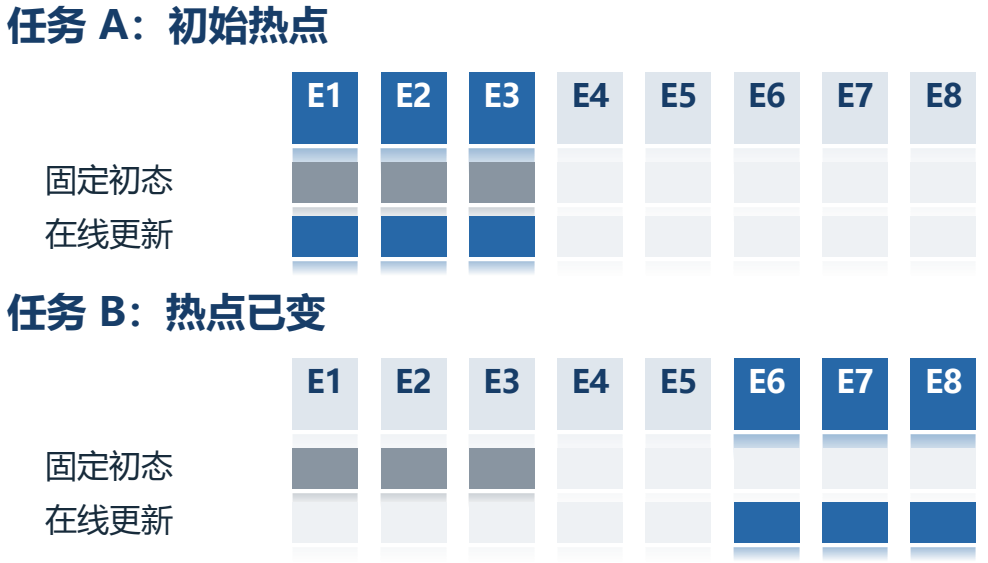
- **部署前画像能预测初始热点，却无法保证后续请求仍选择同一批专家。任务切换后，固定放置继续覆盖旧热点；按真实路由更新的缓存逐渐转向新热点。左图显示切换前后的 SSD 请求比例。**

(a) 任务切换前后的 SSD 读取



第 9 个窗口切换任务；固定初态持续漏掉新热点。

(b) 热点变化怎样造成错位



深蓝：当前常用或已保留的专家

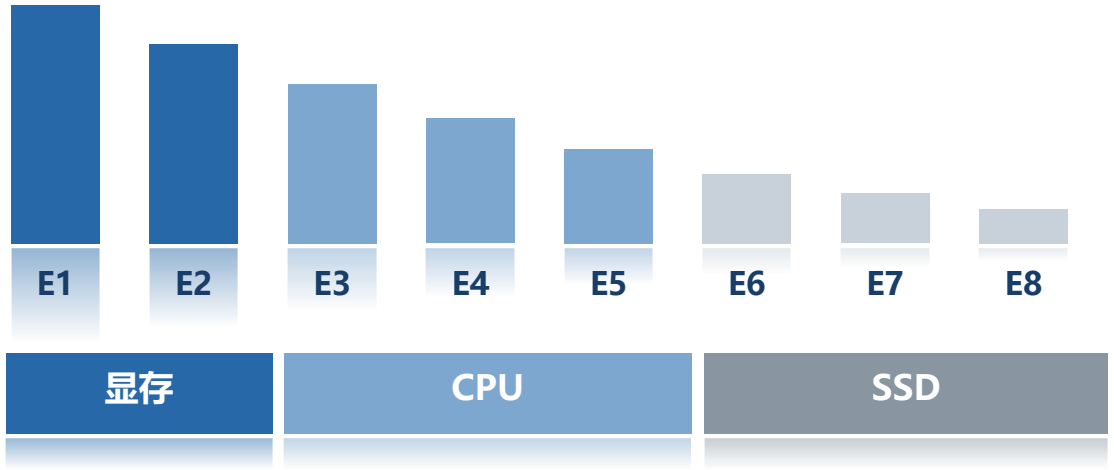
设计问题③：真实路由改变时，怎样修正初始放置？

Insight I: Place Likely Experts Before Serving

- **冷启动时还没有真实请求反馈，初始驻留只能利用部署前的访问画像。MECC 按层估计专家热度，在显存和 CPU 预算内优先放置高概率专家；其余专家保留在 SSD，仍可按需读取。**

机制示意

按画像中的访问频率排序



高频专家放入更快的层级，减少启动段冷读。

为什么能解决对应问题

输入

代表性请求的专家访问频率

决定

每层显存与 CPU 能保留哪些专家

效果

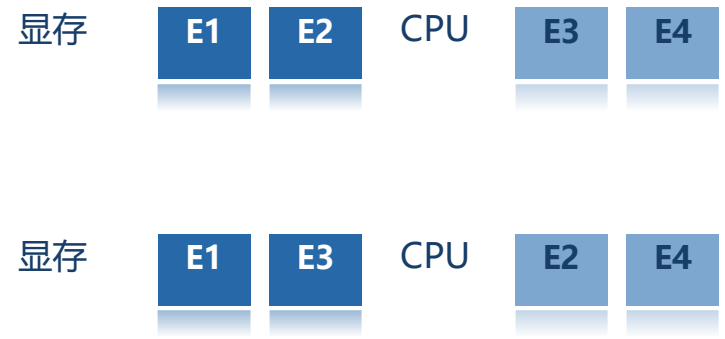
减少启动段落到 SSD 的请求

对应问题①：在首个请求到来前，用离线先验降低冷专家被选中的概率。

Insight II: Retain Experts Across the Two Fast Tiers

- **显存淘汰并不意味着专家不再被用到。MECC 将被替换的专家降到 CPU，并把 CPU 命中的专家提升到显存。两层联合管理保留更多不同专家，使短距离再访可以避开 SSD。**

机制示意



E3 从 CPU 提升到显存；被替换的 E2 降到 CPU。
E2 再次被选中时，无需从 SSD 读取。

为什么能解决对应问题

输入
当前访问与两层驻留状态

决定
命中后升迁；淘汰时降级保留

效果
扩大唯一专家覆盖并承接再访

对应问题②：让有限的显存与 CPU 槽位共同服务于专家复用。

Insight III: Correct the Prior with Real Requests

- **画像是部署前的预测，真实路由是运行时的反馈。MECC 保留初始计划的启动收益，随后用访问序列修正显存与 CPU 驻留，使缓存逐渐覆盖新任务的热点。**

机制示意



初态来自画像；之后的实际路由决定驻留更新。

为什么能解决对应问题

输入

当前任务产生的真实路由

决定

持续更新两层缓存的专家位置

效果

热点切换后逐步恢复快层覆盖

对应问题③：把离线先验与在线反馈放在两个时间尺度上使用。

System Overview: From Profile to Expert Execution

- **MECC 有两个阶段：部署前编译放置计划，服务时根据真实路由访问和更新缓存。**
  SSD 保存完整专家权重；显存和 CPU 只保留其中一部分，权重就绪后才执行所选专家。

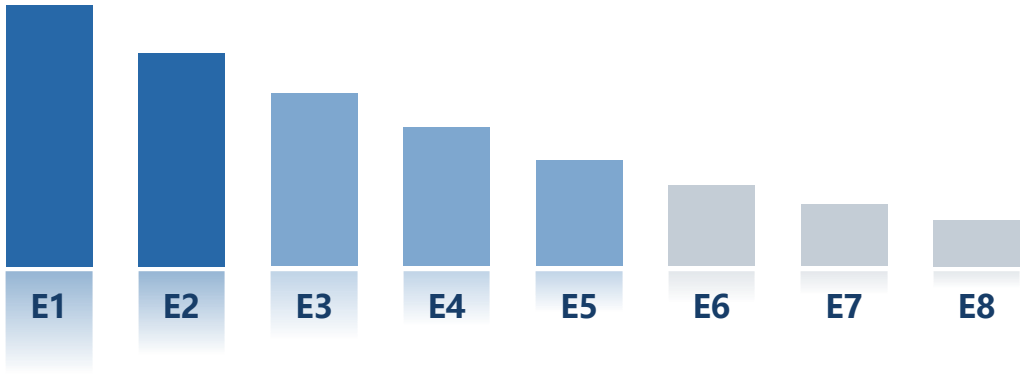


编译期提供起点；运行期不断纠偏；专家数值和路由选择不因搬运而改变。

Method I: Budget-Constrained Placement

- 编译器读取每层专家访问画像、权重大小和单任务显存/CPU 额度。它先决定每层可放多少专家，再按访问热度填入显存、CPU 和 SSD，输出启动时的驻留计划。

输入：按访问频率排序的专家



访问越频繁，冷读的预期代价越高。

输出：每层的初始驻留

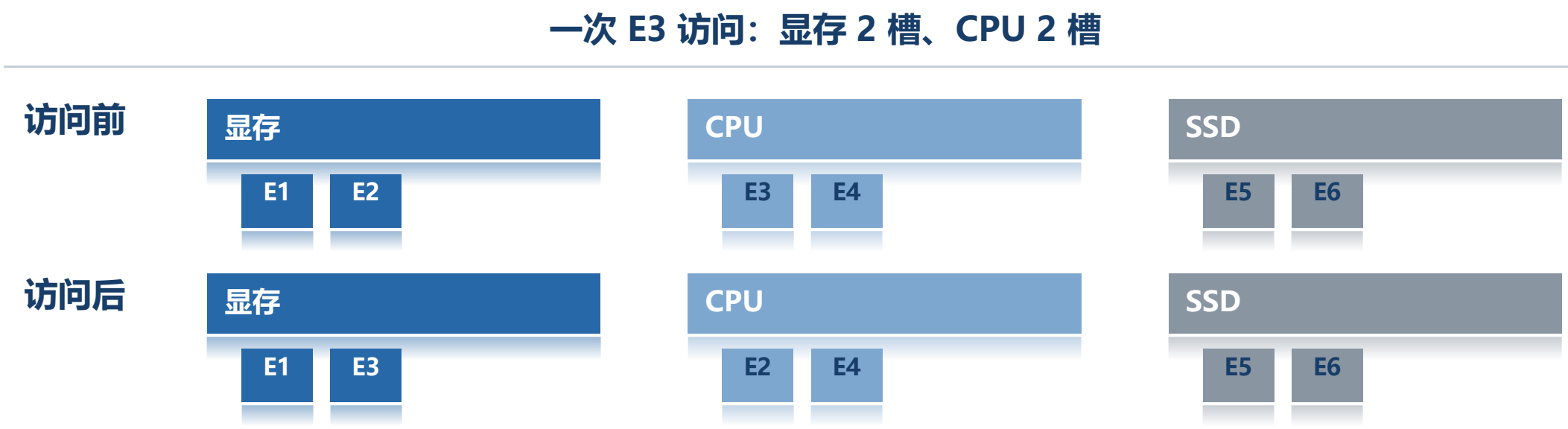


计划规定“先放哪里”；实际访问仍会在运行中改变这些位置。

目标：在给定容量下，降低所选专家的预期取数代价。

Method II: Joint HBM–CPU Retention

- 服务期间，每次路由选择都先查显存，再查 CPU；两层都没有时才读取 SSD。CPU 命中可以提升到显存，被替换的显存专家尽量留在 CPU。下图演示一次访问如何改变驻留状态。

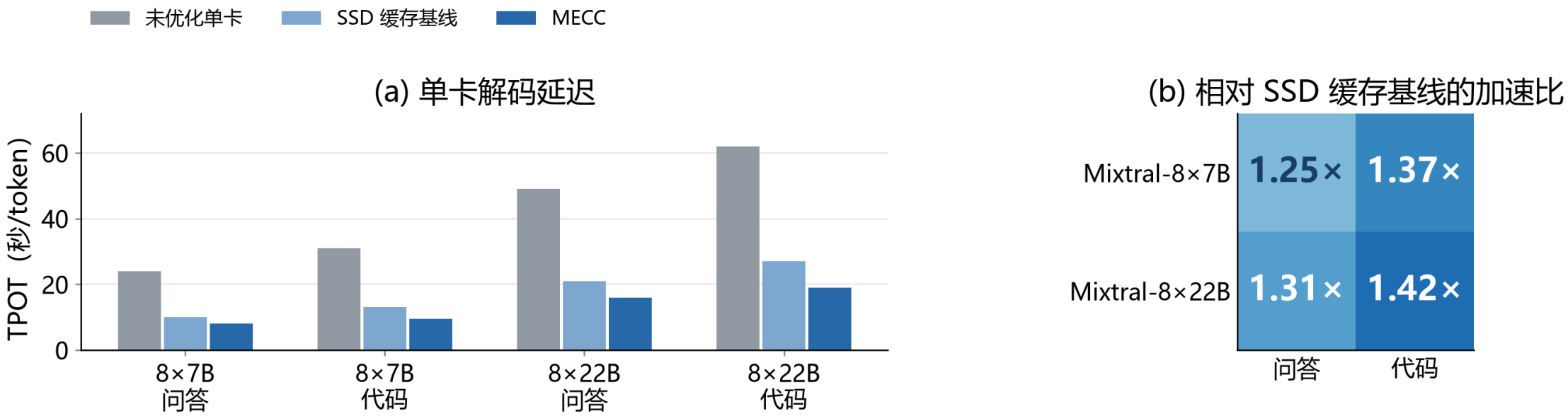


E3 从 CPU 升到显存；E2 降到 CPU。E2 后续再访时仍能避开 SSD。

搬运只改变权重所在位置；路由选择、权重数值与专家计算保持不变。

Results: Single-GPU Decode Latency

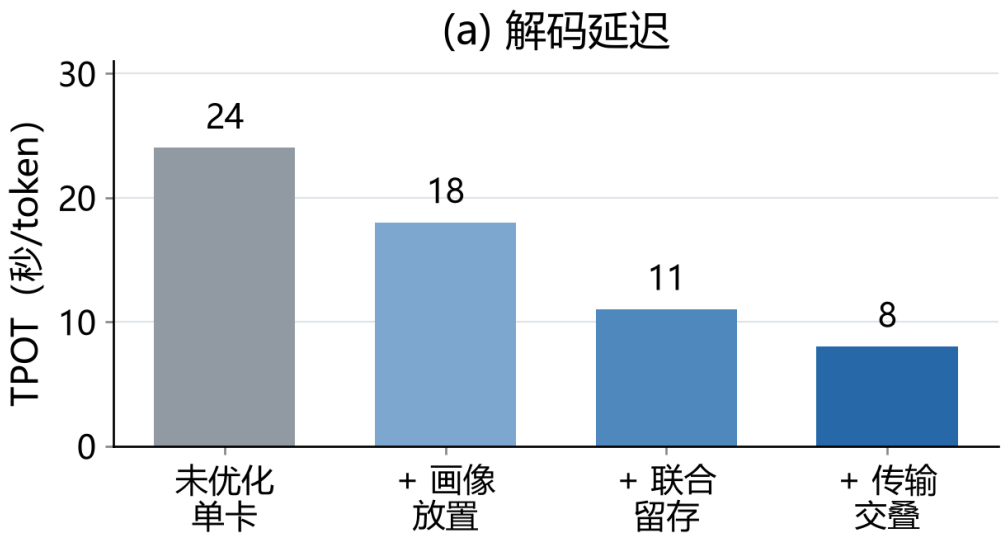
- **在四组模型与任务的理想结果中，MECC 的单卡解码延迟低于未优化流式执行和本地 SSD 缓存基线。左侧柱状图报告每 token 延迟；右侧热图报告相对较接近基线的加速比。**

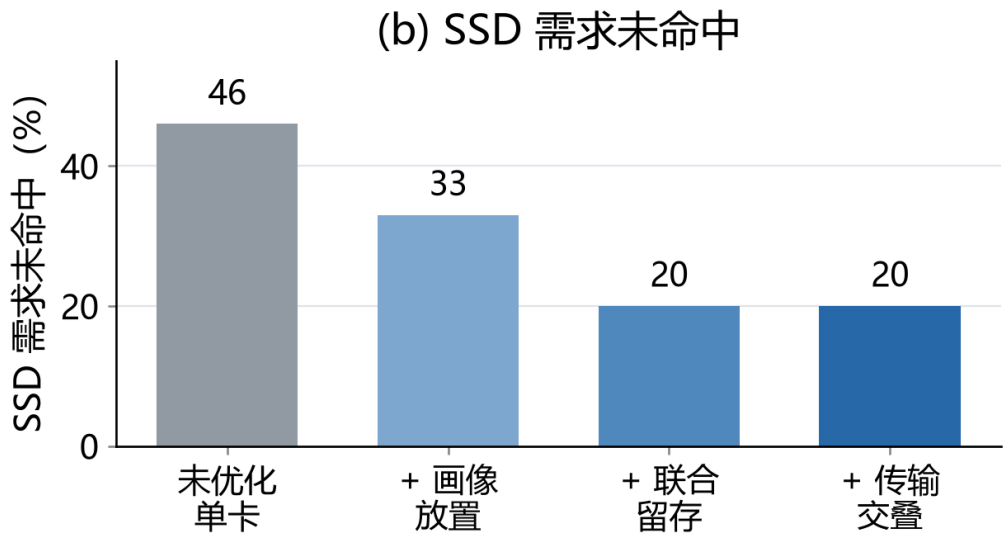


同一任务内存额度下，MECC 与 SSD 缓存基线相差 1.25–1.42×。

Ablation: Placement, Retention, Delivery

- **逐项加入画像放置、跨层联合留存和交叠交付，区分每项设计的贡献。左图显示解码延迟，右图显示真正落到 SSD 的请求比例；只有交付优化进一步减少了暴露等待。**

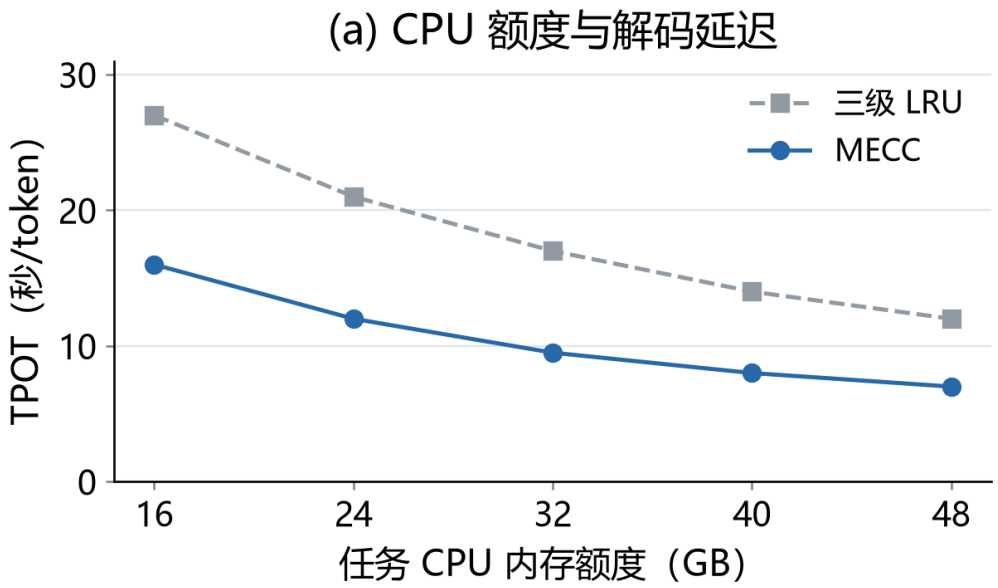


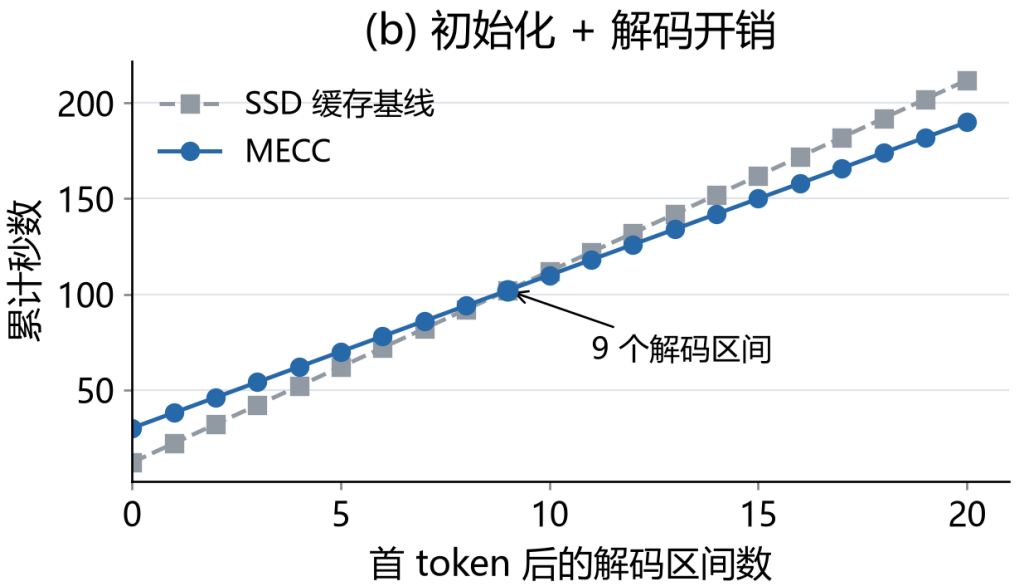


首组任务：24 → 18 → 11 → 8 秒/token；SSD 请求比例：46% → 33% → 20%。

Resource Sensitivity: CPU Cap and Service Length

- **可分配给任务的 CPU 内存越多，能够保留的专家越多；服务越长，启动时的画像和加载开销越容易被后续 token 摊薄。两张图分别改变内存额度和解码时域。**

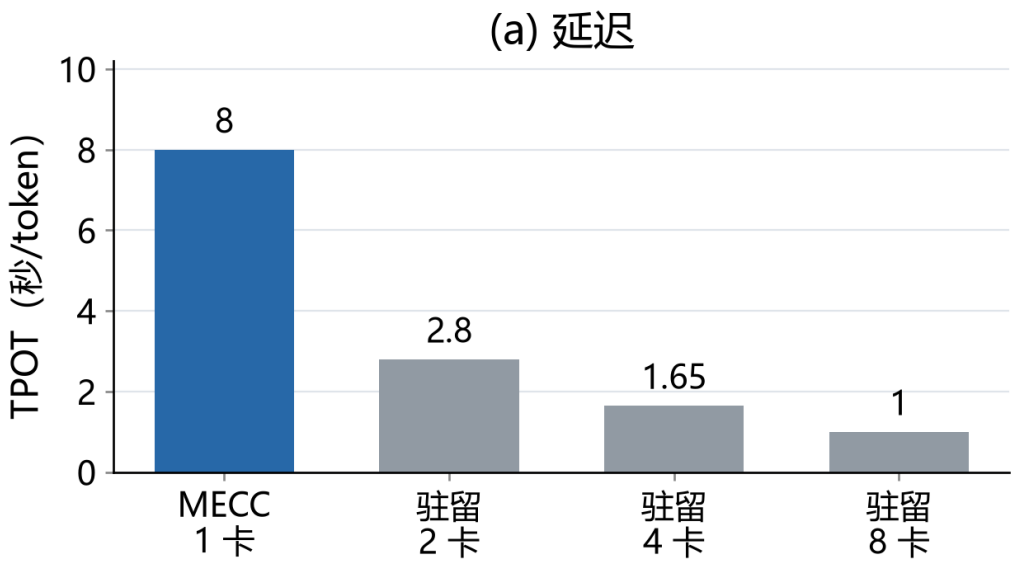


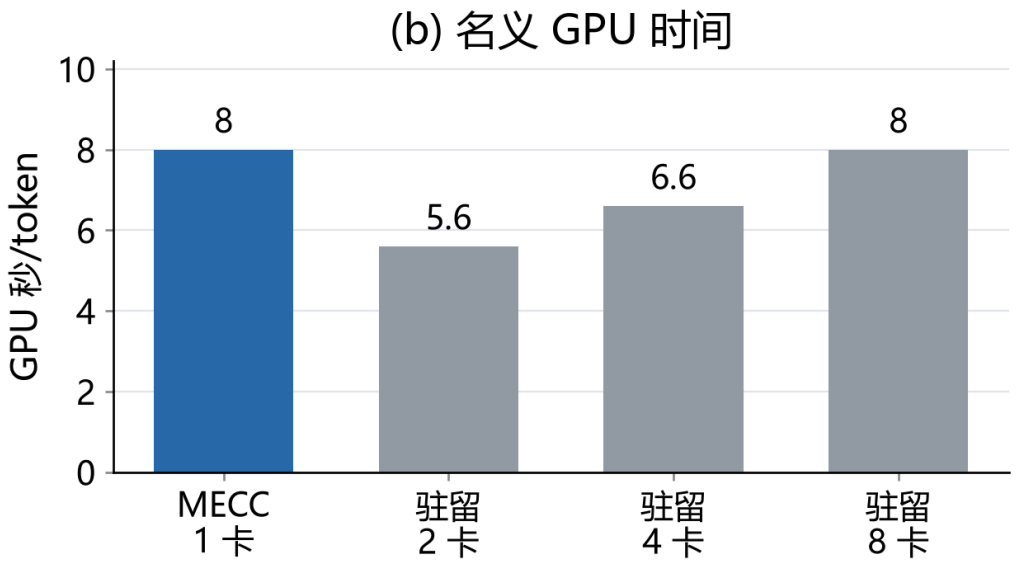


部署选择同时取决于 CPU 额度和请求长度。

One GPU versus Resident Multi-GPU

- **完整驻留的多卡方案拥有更短的单 token 延迟；MECC 用一张 GPU 执行完整模型。右图把 GPU 张数乘以延迟，仅作为名义 GPU 时间的比较。**





单卡方案适用于 GPU 配额受限、能够接受较长单请求延迟的任务。

Conclusion: Single-GPU SSD-Active MoE

- **当单任务 CPU 内存不能容纳全部专家时，SSD 会进入 MoE 推理路径。MECC 通过部署前放置、运行中保留和使用前交付三个决定，减少冷专家访问造成的等待。**

放置

画像确定初始位置

回应冷启动

留存

显存与 CPU 联合更新

回应重复冷读

纠偏

按真实路由修正驻留

回应热点漂移

面向一张 GPU、有限 CPU 额度、允许一定响应延迟的完整 MoE 部署。